



Universidad Nacional de La Matanza
Departamento de Ingeniería e Investigaciones Tecnológicas

ACTIVIDAD PRÁCTICA OBLIGATORIA N°1- PROGRAMACIÓN CONCURRENTES - 2C 2025

Mathieu Andres Santamaría Loiacono

42.823.461

msantamarialoiacono@alumno.unlam.edu.ar

Alejo Lopez Rodofile

42.193.614

alopezrodofile@alumno.unlam.edu.ar

Martín Alejandro Didolich

43.664.688

madidolich@alumno.unlam.edu.ar

Fabicio Javier Martínez Solomita

43.871.283

fmartinezsolomita@alumno.unlam.edu.ar



Universidad Nacional de La Matanza

Departamento de Ingeniería e Investigaciones Tecnológicas

Resumen

El auge de la Inteligencia Artificial Generativa (IAG) ha transformado las metodologías de desarrollo de software, introduciendo el concepto de Vibecoding, en el que los desarrolladores emplean Large Language Models (LLMs) para generar código automáticamente a partir de instrucciones en lenguaje natural. Este cambio de paradigma requiere evaluar críticamente la calidad, eficiencia y robustez del código generado, especialmente en dominios complejos como la programación concurrente, donde la correcta implementación de hilos y la sincronización son fundamentales para evitar errores y garantizar un paralelismo eficiente. En este trabajo se investiga la capacidad de tres herramientas de generación de código —ChatGPT GPT-5, Gemini 2.5 Flash y Copilot Claude Sonnet 4— para implementar la multiplicación de matrices concurrente en Python. Se diseñó un prompt técnico que solicitaba delegar la computación de submatrices a hilos, cumpliendo con criterios de claridad, modularidad y buenas prácticas de programación. El código generado fue analizado en términos de ejecución concurrente, sincronización, eficiencia temporal, legibilidad y mantenibilidad. Los resultados muestran que todas las herramientas producen código funcional y concurrente, aunque se observan diferencias en eficiencia y estructuración: ChatGPT destacó en velocidad de ejecución y sincronización robusta, Gemini ofreció buen equilibrio entre claridad y paralelismo, y Copilot necesitó ajustes menores para generar un bloque principal ejecutable. Este estudio evidencia las ventajas de la IAG para acelerar tareas computacionales y las limitaciones que aún existen en la generación automática de código concurrente robusto, proporcionando un marco para futuras investigaciones en entornos académicos y profesionales.

1. Introducción

En los últimos años, el desarrollo de software ha experimentado una transformación profunda impulsada por la Inteligencia Artificial Generativa (IAG). La capacidad de los modelos de lenguaje a gran escala (LLMs, por sus siglas en inglés) para producir código a partir de instrucciones en lenguaje natural ha alterado el papel del desarrollador, desplazándolo del rol de creador principal hacia funciones de verificación, validación e integración del código generado. Este fenómeno, recientemente bautizado como *vibe coding* o *Vibecoding*, propone un nuevo paradigma en el que la programación resulta más accesible, pero suscita preguntas críticas respecto a la calidad, robustez y eficiencia del software producido [1].

Si bien esta técnica promete acelerar la creación de software, plantea interrogantes críticos respecto a la calidad, robustez y eficiencia del código, especialmente en contextos de programación concurrente, donde la correcta implementación de hilos, sincronización y gestión de

recursos es fundamental. Errores en estos escenarios pueden derivar en condiciones de carrera, interbloqueos o resultados incorrectos, comprometiendo la fiabilidad del sistema. [2].

En este estudio, se centra la atención en un caso clásico de computación concurrente: la multiplicación de matrices en Python utilizando hilos. Este problema representa un desafío adecuado para evaluar la capacidad de las herramientas de Vibecoding, al requerir la asignación eficiente de tareas a múltiples hilos, la sincronización adecuada y la escritura de código claro y modular.

El objetivo de la investigación es analizar comparativamente el desempeño de tres herramientas de IAG —ChatGPT GPT-5, Gemini 2.5 Flash y Copilot Claude Sonnet 4— en la generación de código concurrente. Se evaluará no solo la funcionalidad y eficiencia del código producido, sino también su claridad, mantenibilidad y adecuación a buenas prácticas de programación. Este enfoque combina la evaluación cuantitativa mediante métricas de rendimiento y calidad de software con un análisis cualitativo de la estructura y legibilidad del código, estableciendo una base sólida para futuras investigaciones sobre la aplicación de IAG en entornos académicos y profesionales.

1.1 Vibecoding y los retos de la calidad del código

El término *Vibecoding* fue introducido por Andrej Karpathy en 2025 para describir un enfoque en el que el desarrollador “se deja llevar por las vibraciones”, entregando casi por completo la escritura del código al modelo, sin revisiones profundas [3]. Según esta concepción, el humano interactúa mediante prompts conversacionales, pruebas e iteraciones en lugar de programar directamente. El modelo genera gran parte del esqueleto del software, y el desarrollador actúa más como orquestador y validador que como autor del código.

Aunque esta visión promete democratizar la creación de software, conlleva desafíos importantes:

Vulnerabilidades y seguridad: el código generado puede incluir errores lógicos o fallas de seguridad inadvertidas [4].

Mantenibilidad: aunque el código sea sintácticamente correcto, puede carecer de una arquitectura coherente, modularidad o claridad que favorezcan su posterior evolución [5].

Corrección y rendimiento: en dominios complejos como el paralelismo, el modelo puede producir código funcional pero poco óptimo, o incluso incorrecto en cuanto a sincronización y eficiencia [6].



Universidad Nacional de La Matanza

Departamento de Ingeniería e Investigaciones Tecnológicas

Descompensación de responsabilidad: al depender del modelo para generar gran parte del código, el desarrollador corre el riesgo de perder comprensión sobre las decisiones internas [7].

Para subsanar estas debilidades metodológicas, este estudio adopta un enfoque de evaluación basado en la funcionalidad, eficiencia y calidad estructural del código generado por herramientas de IAG [8]. Se analiza cómo cada modelo maneja la asignación de tareas concurrentes, la sincronización de hilos y la modularidad del código, permitiendo identificar fortalezas y limitaciones en la producción de software robusto y legible.

Si bien existen múltiples criterios para evaluar código generado automáticamente, la literatura reciente sugiere que métricas de desempeño y observaciones sobre la correcta implementación de paralelismo pueden ser indicativas de la confiabilidad del código [9]. Sepidband et al. demuestran que un análisis sistemático del comportamiento del código concurrente permite mejorar la tasa de éxito en tareas complejas de programación generada por LLMs [10]. Asimismo, benchmarks como DynaCode ofrecen criterios de evaluación orientados al rendimiento y estructura del código, facilitando la comparación entre modelos según su comportamiento en tareas con distintos niveles de dificultad [11].

De esta manera, el presente estudio valida el uso de herramientas de IAG para la programación concurrente, proponiendo una metodología que combina la revisión de funcionalidad, eficiencia y claridad del código, contribuyendo a comprender mejor el rol del desarrollador en la era del *Vibecoding*.

2. Desarrollo

Actualmente, el ecosistema de herramientas de Inteligencia Artificial Generativa (IAG) ofrece múltiples alternativas para la asistencia en tareas de programación. Desde asistentes integrados en editores de código hasta plataformas conversacionales, estas soluciones han

ampliado la capacidad de los desarrolladores para producir, probar y refinar software a partir de descripciones en lenguaje natural. En este estudio, el foco se centra en herramientas de *Vibecoding* que permiten la generación de código en Python, con énfasis en problemas de programación concurrente, específicamente la multiplicación de matrices utilizando hilos.

2.1 Herramientas seleccionadas

Para llevar adelante la investigación, se seleccionaron tres herramientas representativas del estado actual del *Vibecoding*:

ChatGPT (GPT-5, versión gratuita): desarrollado por OpenAI, este asistente conversacional ofrece generación de

código en múltiples lenguajes, con soporte para Python [12]. En su modalidad no premium, permite elaborar scripts completos, pero presenta limitaciones en la generación de proyectos estructurados o archivos múltiples. Su principal fortaleza radica en la claridad explicativa, que facilita comprender el funcionamiento del código generado.

Gemini 2.5 Flash: desarrollado por Google DeepMind, es un modelo optimizado para velocidad y consultas técnicas generales [13]. Si bien está orientado a un uso más amplio (desde generación de texto hasta soporte en aplicaciones móviles), puede emplearse para *Vibecoding* de manera parcial. Una de sus ventajas es la rapidez de respuesta, aunque los resultados suelen ser menos detallados en cuanto a manejo de concurrencia avanzada en Python.

Copilot Claude Sonnet 4: este asistente, integrado en entornos de desarrollo, ofrece generación de código contextualizado a partir del análisis del archivo en edición [14]. A diferencia de las soluciones conversacionales, su fortaleza está en la asistencia directa y dinámica durante la escritura de código, sugiriendo fragmentos que se adaptan al flujo de trabajo. En el contexto de esta investigación, resultó útil para refinar implementaciones concurrentes y mejorar la legibilidad del código generado.

En el desarrollo de software asistido por inteligencia artificial, el diseño del prompt adquiere un rol central, ya que de él depende en gran medida la calidad, precisión y reproducibilidad del código generado. No se trata únicamente de indicar una tarea a la herramienta, sino de construir una instrucción lo suficientemente clara y estructurada como para guiar el proceso de generación hacia un resultado coherente con los objetivos planteados.

En este contexto, resulta útil diferenciar entre dos enfoques. Por un lado, el prompt básico, que corresponde a una instrucción redactada de manera libre y sin una estructura rígida, similar a cómo un programador explicaría un problema de forma espontánea. Este tipo de prompt puede ser suficiente en escenarios simples, pero tiende a producir salidas heterogéneas y, en ocasiones, ambiguas.

Por otro lado, se encuentra el prompt técnico, diseñado bajo principios más cercanos a una especificación formal. Este enfoque organiza la instrucción en secciones fijas que delimitan con claridad el lenguaje de programación, la plataforma, las funcionalidades a implementar, los requisitos específicos y las restricciones. Su propósito es minimizar la ambigüedad, facilitar la reproducibilidad y generar código más consistente con las necesidades planteadas.

En las secciones siguientes se presentarán ambos enfoques: primero el prompt básico redactado manualmente, y luego el prompt técnico, que constituye la base metodológica utilizada para las pruebas de generación de código en este trabajo.



Universidad Nacional de La Matanza

Departamento de Ingeniería e Investigaciones Tecnológicas

Para el caso en estudio se presentaron los siguientes requisitos.

“Se quiere generar el código en python para desarrollar un programa concurrente que pueda resolver una multiplicación de matrices, se debe realizar una función que reciba como parámetro las 2 matrices y usando programación concurrente sea capaz de resolver la multiplicación en caso de que sea posible realizando las validaciones necesarias. Se debe realizar usando hilos si o si y también es importante que se use la cantidad de hilos como filas/columnas haya en las matrices a multiplicar. El código debe ser en inglés, no debe haber magic numbers, se debe usar Coddling-Standard de google y las funciones no deben tener más de 15 líneas.”

En base a ello, se definió el prompt técnico de la siguiente manera.

- LENGUAJE DE PROGRAMACION:
 - Utilizar Python.
- PLATAFORMA
 - Windows.
- FUNCIONALIDAD A IMPLEMENTAR:
 - Desarrollar un programa concurrente para realizar la multiplicación de dos matrices
- REQUISITOS ESPECIFICOS:
 - Se debe realizar una función que reciba las 2 matrices como parámetros.
 - Se debe usar programación concurrente.
 - Si o si deben utilizarse hilos.
 - Se deben usar la misma cantidad de hilos como filas/columnas haya en las matrices a multiplicar.
- RESTRICCIONES:
 - El código debe ser en inglés.
 - No debe haber magic numbers en el código.
 - Se de usar Coddling-Standard de google.
 - Las funciones no deben tener más de 15 líneas.

Resultados obtenidos en ChatGPT

Se generó un código aceptable, implementando correctamente el pedido realizado, cumpliendo así con los requisitos propuestos. Tiene la capacidad de ejecutarse para cualquier matriz planteada, no presenta inconsistencias ni errores al momento de la ejecución, no fue necesario realizar nuevas iteraciones para que pueda ser ejecutado, se tuvieron en cuenta las restricciones planteadas, no hubo necesidad de utilizar otra IA para obtener el resultado y no se debieron realizar refinamientos.

Resultados obtenidos en Gemini

Al igual que la IA anteriormente mencionada, el código generado resultó satisfactorio, ya que implementa de manera correcta lo solicitado y se ajusta a los requisitos planteados. La solución funciona con cualquier tamaño de

matriz sin presentar errores en la ejecución. Tampoco fue necesario realizar iteraciones adicionales ni utilizar otra IA, dado que se respetaron las restricciones definidas y no se requirieron ajustes posteriores.

Resultados obtenidos en GitHub Copilot

En la primera iteración, la IA generó únicamente la función de multiplicación de matrices considerando los hilos, pero no incluyó un bloque `main` para ejecutar el programa. Por esta razón, fue necesaria una segunda iteración para que el código funcionara correctamente. A pesar de esto, la solución cumplió con los requisitos y respetó las restricciones planteadas.

2.2 Métricas de evolución

Para evaluar el desempeño de las herramientas de Inteligencia Artificial Generativa (ChatGPT GPT-5, Gemini 2.5 Flash y Copilot Claude Sonnet 4) en la generación de código para multiplicación de matrices concurrente en Python, se aplicarán métricas específicas que permiten analizar tanto el rendimiento de ejecución como la calidad del código generado. Estas métricas proporcionan una base objetiva para determinar cuál de las IAs produce soluciones más eficientes, robustas y legibles.

Métricas de Rendimiento y Eficiencia

Estas métricas se centran en la ejecución del código y la correcta gestión de la concurrencia:

a. Tiempo de Finalización de la Tarea:

Mide el tiempo total que tarda en ejecutarse la multiplicación de matrices concurrente generada por cada herramienta. Permite comparar la rapidez de ejecución y la eficiencia en el manejo de hilos.

b. Eficiencia de Sincronización:

Evalúa cómo se gestionan los hilos y la coordinación entre ellos para evitar condiciones de carrera y optimizar el paralelismo. Se analiza la correcta implementación de `thread.join()` y la asignación de tareas a cada hilo, considerando el impacto del overhead en matrices de diferentes tamaños.

Métricas de Calidad y Precisión del Código

Estas métricas se enfocan en la claridad, mantenibilidad y estructura del código generado:

a. Legibilidad del Código:

Se analiza la claridad estructural, el uso de nombres descriptivos para funciones y variables, la presencia de

docstrings y comentarios, así como la organización general del código.

b. Mantenibilidad del Código:

Evalúa la modularidad, la facilidad para realizar cambios



Universidad Nacional de La Matanza

Departamento de Ingeniería e Investigaciones Tecnológicas

o agregar optimizaciones, y la posibilidad de adaptar el código a diferentes estrategias de concurrencia sin afectar la lógica central.

3. Resultados

Tiempo de Finalización de la Tarea

Se evaluó el tiempo requerido por cada herramienta de Inteligencia Artificial Generativa para completar la multiplicación de matrices concurrente en Python. Los resultados obtenidos fueron los siguientes:

ChatGPT (GPT-5): 0.7746 ms

Gemini 2.5 Flash: 0.824 ms

Copilot Claude Sonnet 4: 0.9244 ms

De acuerdo con estos resultados, ChatGPT demostró la mayor eficiencia temporal, seguido por Gemini, mientras que Copilot presentó un tiempo ligeramente superior. Esta métrica permite comparar la agilidad de cada herramienta en la ejecución de operaciones concurrentes de cómputo intensivo.

Eficiencia de Sincronización

La eficiencia de sincronización se analizó observando cómo cada herramienta implementa la concurrencia mediante hilos en Python y el uso de `thread.join()` para la coordinación entre hilos:

ChatGPT (GPT-5): Cada fila de la matriz se calcula en un hilo independiente, con sincronización mediante `thread.join()`, asegurando que el hilo principal espere la finalización de todos los hilos. Este enfoque previene condiciones de carrera y permite paralelismo moderado; no obstante, para matrices de gran tamaño, el overhead de creación y unión de hilos puede afectar ligeramente la eficiencia.

Gemini 2.5 Flash: La multiplicación se gestiona de manera análoga, asignando un hilo independiente por fila con `_calculate_row` y sincronización mediante `thread.join()`. Se evita la concurrencia conflictiva y se mantiene un paralelismo adecuado para cargas pequeñas a medianas.

Copilot Claude Sonnet 4: Cada hilo calcula una fila de la matriz utilizando `_calculate_row`, sincronizándose mediante `thread.join()`. Esta estrategia evita condiciones de carrera y permite paralelismo moderado, aunque el overhead de los hilos puede ser notable en matrices de gran tamaño.

Legibilidad y Mantenibilidad del Código

La calidad del código se evaluó considerando la claridad, modularidad y facilidad de mantenimiento:

ChatGPT (GPT-5): El código es claro y modular, con funciones bien delimitadas (`multiply_row` y `multiply_matrices`) y nombres descriptivos que facilitan su comprensión. La estructura secuencial del `main()` permite seguir el flujo de ejecución de manera intuitiva, y la modularidad garantiza que futuras modificaciones puedan realizarse sin alterar la lógica central.

Gemini 2.5 Flash: El código presenta una organización modular y legible, separando la validación de matrices (`_validate_matrices`), el cálculo de filas (`_calculate_row`) y la gestión de hilos (`threaded_matrix_multiplication`). Los docstrings explicativos y un `main()` lineal refuerzan la comprensión, permitiendo extender o modificar la funcionalidad sin comprometer la estructura principal.

Copilot Claude Sonnet 4: La estructura del código es clara y modular, con separación de validación, cálculo de filas y gestión de hilos. Los nombres descriptivos y la documentación mejoran la legibilidad, mientras que la modularidad facilita la implementación de optimizaciones o cambios en la estrategia de concurrencia sin afectar la lógica central.

4. Conclusiones

En esta investigación se llevó a cabo un análisis comparativo del desempeño de tres herramientas de Inteligencia Artificial Generativa (ChatGPT GPT-5, Gemini 2.5 Flash y Copilot Claude Sonnet 4) en la generación de código concurrente en Python para la multiplicación de matrices. Se evaluaron métricas de rendimiento y eficiencia, como el tiempo de finalización de la tarea y la eficiencia de sincronización, así como métricas de calidad y precisión del código, enfocadas en la legibilidad y mantenibilidad.

Los resultados obtenidos indican que ChatGPT presentó el menor tiempo de ejecución y una correcta implementación de la sincronización mediante hilos, destacándose por su eficiencia temporal. Gemini mostró un desempeño intermedio, manteniendo buena modularidad y claridad en el código, mientras que Copilot, aunque funcional y modular, requirió una iteración adicional para incluir un bloque `main`, lo que afectó ligeramente su desempeño inicial.

En términos de legibilidad y mantenibilidad, las tres herramientas generaron código modular y comprensible, facilitando su adaptación y extensión futura. No obstante, se observan diferencias sutiles en la claridad de la documentación y la estructuración de funciones, que podrían influir en la facilidad de uso en proyectos de mayor envergadura.

Estos hallazgos permiten concluir que, si bien todas las herramientas evaluadas cumplen con los requisitos funcionales, ChatGPT se destaca en eficiencia temporal y robustez de la implementación concurrente, mientras que Gemini y Copilot ofrecen ventajas comparativas en claridad



Universidad Nacional de La Matanza

Departamento de Ingeniería e Investigaciones Tecnológicas

y modularidad del código. La investigación establece así un marco de referencia para futuras evaluaciones de herramientas de IAG en tareas de programación concurrente, contribuyendo a la comprensión de sus fortalezas y limitaciones en entornos académicos y de desarrollo profesional.

[14] Anthropic. (2025). Claude Sonnet 4 Copilot Integration Guide. Disponible en: <https://www.anthropic.com/claude-sonnet>

Referencias

[1] Alfonseca, E., et al. (2024). Large Language Models and Code Generation: A Survey. *ACM Computing Surveys*, 57(1), 1–39.

[2] Herlihy, M., & Shavit, N. (2021). *The Art of Multiprocessor Programming* (Revised Reprint). San Francisco, CA: Morgan Kaufmann.

[3] Karpathy, A. (2025). Vibe coding. *X (Twitter)*.

[4] Pearce, N., Kolter, J. Z., & Christian, A. (2023). Examining the Security Risks of AI-Coded Software. En *Proceedings of IEEE S&P Workshops*, 345–352.

[5] Vaithilingam, S., Zhang, T., & Glassman, E. L. (2022). Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. En *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 1–12.

[6] Chen, P., et al. (2022). Competition-Level Code Generation with AlphaCode. *Science*, 375(6585), 843–848.

[7] Sarkar, A., et al. (2020). What's Wrong with Code Reviews? An Empirical Study of Code Review Practices in Industry. En *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, 1411–1422.

[8] McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320.

[9] Kemerer, C. F. (1991). A Longitudinal Empirical Study of the Validity of McCabe's Cyclomatic Complexity Measure. *IEEE Transactions on Software Engineering*, 17(12), 1213–1227.

[10] Sepidband, A., et al. (2025). Measuring Code Complexity for Improving AI Code Generation.

[11] Liu, Y., et al. (2025). DynaCode: A Benchmark for Evaluating LLMs on Dynamic Code Generation Tasks.

[12] OpenAI. (2025). ChatGPT (GPT-5) Technical Documentation. Disponible en: <https://openai.com/research/gpt-5>

[13] Google DeepMind. (2025). Gemini 2.5 Flash User Guide. Disponible en: <https://www.deepmind.com/research/gemini-2-5>